

The TEMPO Scheduling Model

A formal specification of resident clinic availability, capacity and day-construction for an Internal Medicine continuity clinic

ACADEMIC YEAR 2026–27 · 40 RESIDENTS · 12 BLOCKS · 261 CLINIC WEEKDAYS

The scheduling problem is a **constrained assignment under partial information**. Who is in clinic on a given date is stored nowhere — it is *derived*, and for much of the year the data it is derived from is genuinely undecided. The model makes that gap explicit rather than defaulting over it.

§1 Objects and the block map

R is the set of residents, $|R| = 40$, split evenly by level $\ell : R \rightarrow \{\text{PGY-1, PGY-2}\}$. $B = \{1, \dots, 12\}$ are the rotation blocks. D are the dates of the academic year $Y = [2026-07-01, 2027-06-30]$ — 365 days holding **261 clinic weekdays**. $W = \{1, \dots, 5\}$ are Monday to Friday, and $w(d)$ is the weekday of date d .

Each block is a closed date interval $[s_k, e_k]$. The twelve intervals are pairwise disjoint and contiguous, so they **partition** the year:

$\beta : D \rightarrow B \cup \{\perp\}$, $\beta(d) = k \iff s_k \leq d \leq e_k$, otherwise $\perp$ (outside the year)

Length check — 7 blocks of 28 days + 4 blocks of 35 days + 1 block of 29 days = 196 + 140 + 29 = **365**, exactly the span of Y .
No gaps, no overlaps.

1.1 The rotation matrix and the elective-day map

Every resident holds one rotation per block, encoded as a letter — a total function, a dense $40 \times 12 = \mathbf{480\text{-entry}}$ **matrix** supplied by the residency programme:

$C : R \times B \rightarrow \{\text{O, N, E, A}\}$

O outpatient — clinic every weekday of the block · **N** inpatient / nights / ICU — no clinic · **E** elective-type — exactly one clinic weekday per week · **A** away rotation — as E, flagged for confirmation

Codes **E** and **A** fix the *frequency* — one day a week — but not *which* day. That is a second, independent datum, and critically a **partial** function, since the programme settles it only about a month ahead:

$\delta : R \times B \rightarrow W \cup \{\perp\}$, with $\delta(r, k) = \perp$ whenever the clinic day for that block is undecided

The domain of δ is entered by the scheduler and is empty on first use. **No value of δ is ever inferred, defaulted or interpolated.**

§2 The availability function

Everything the application displays follows from one function. Writing $k = \beta(d)$:

$a(r, d) =$

NONE if $k = \perp$, or $w(d) \notin W$, or r is inactive (outside year, weekend, off roster)
FULL if $C[r, k] = \text{O}$
NONE if $C[r, k] = \text{N}$
ELECTIVE_TBD if $C[r, k] \in \{\text{E, A}\}$ and $\delta(r, k) = \perp$
ELECTIVE_DAY if $C[r, k] \in \{\text{E, A}\}$ and $w(d) = \delta(r, k)$
NONE otherwise

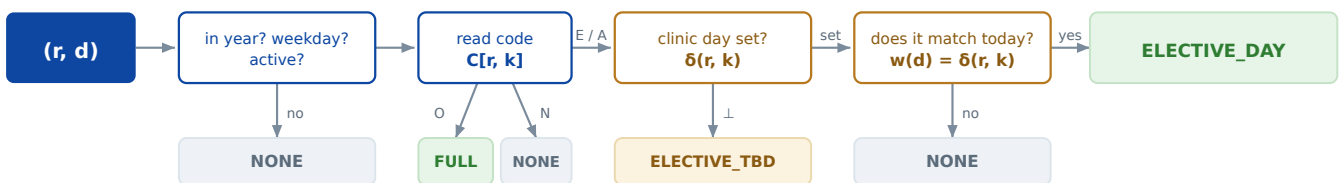


Figure 1 — The availability decision. Four outcomes, not three. The amber terminal is the one most scheduling tools omit: it records that the answer is *not yet knowable*, which is different from the resident being unavailable.

THE SOUNDNESS PROPERTY

Define the in-clinic predicate $\iota(r, d) = [a(r, d) \in \{\text{FULL, ELECTIVE_DAY}\}]$ and the roster $R(d) = \{r : \iota(r, d)\}$.

Because $\perp$ maps to its own outcome instead of collapsing into a weekday, **ι holds only where a human supplied the day**. So $R(d)$ is a **lower bound** on who is truly in clinic, never an over-estimate, and it is **monotone in information**: setting $\delta(r, k) = j$ adds r to exactly the dates of block k with weekday j , and removes them from none. Under-booking is recoverable; double-booking a resident who is on the wards is not.

§3 Templates and horizon-dependent admissibility

A clinic day is not a bucket of appointments but an ordered sequence of typed slots:

$T(\ell, w) = \langle (m_i, \mu_i, c_i) \rangle$, m = start minute, μ = duration, $c \in \{\text{Est, New, FROZEN, HospFU, Any}\}$

Invariant — no slot satisfies $720 \leq m < 780$. The lunch hour is structurally absent, not merely blocked. PGY-2 slots have $\mu = 30$, PGY-1 slots $\mu = 60$. Durations are never shortened to create room.

3.1 Template sizes |T|

Level	Mon–Wed	Thu	Fri
PGY-2 (30 min)	16	8	14
PGY-1 (60 min)	8	4	7

Thursday is a half day, so the count halves; Friday starts at 09:00. Both are structural — they follow from the template, so every downstream number inherits them automatically.

3.2 Class composition, PGY-2 Mon–Wed

Class	n	Admits
Est	9	follow-ups, any horizon
New	4	new patients while $h \geq 14$
FROZEN	2	urgent / discharge, $h \leq 2$
HospFU	1	discharge follow-up ≤ 14 d
Total	16	

Let $h = d - \text{today}$ be the **booking horizon** and v the visit type. Admissibility is a relation on (class, visit, horizon), so the *usable* template changes as a date approaches even though $|T|$ never does:

$\text{adm}(\text{New}, v, h) = (v = \text{NEW} \wedge h \geq 14) \vee (v = \text{EST} \wedge h < 14)$ $\text{adm}(\text{FROZEN}, v, h) = h \leq 2 \wedge v \in \{\text{HOSP}, \text{EST}\}$

A New slot does not vanish at $h = 14$ — it **reclassifies** into an Est slot. A Frozen slot is invisible until $h = 2$, then opens only to urgent, discharge or waitlist work.

3.3 Admissible slot count for one PGY-2 Monday, as a function of horizon

Visit type v	$h \geq 14$	$2 < h < 14$	$h \leq 2$	Reading
Follow-up (EST)	9	13	15	New slots convert, then Frozen releases
New patient (NEW)	4	0	0	the protected window closes at 14 days
Hospital follow-up (HOSP)	1	1	3	the dedicated 13:00 slot, plus Frozen late
PGY-1, any type	8	8	8	class <i>Any</i> is horizon-invariant

§4 The constraint hierarchy

With $b(r, d)$ the patients booked with r on d , and A attendings, four ceilings act at once — three per resident, one across the day:

Per resident $b \leq \kappa(\ell, w) = \lfloor \rho \cdot \tau(\ell, w) \rfloor$ (advance bookings, $\rho = 0.70$) $\leq \tau(\ell, w)$ (target) $\leq |T(\ell, w)|$ (hard maximum)

Across the day $\sum_r b(r, d) \leq \Sigma(d) = A \cdot p(w)$, $p = 32$ (16 on Thursday) $\implies \text{bookable}(d) = \min(\sum_r \tau, \Sigma(d))$

The chain $\kappa \leq \tau \leq |T|$ holds for all six (ℓ, w) combinations under the shipped defaults; the reserve $\tau - \kappa$ is what pays for Frozen releases, hospital follow-ups and same-day flex.

Level / weekday	κ ceiling	τ target	T max	reserve $\tau - \kappa$	slack $ T - \tau$
PGY-2 Mon–Wed	8	12	16	4	4
PGY-2 Thu (AM)	4	6	8	2	2
PGY-2 Fri (09:00)	7	10	14	3	4
PGY-1 Mon–Wed	3	5	8	2	3
PGY-1 Thu	2	3	4	1	1
PGY-1 Fri	2	4	7	2	3

A · ONE RESIDENT — PGY-2, Monday



B · THE WHOLE DAY — Monday 31 Aug 2026, 6 residents, 3 attendings

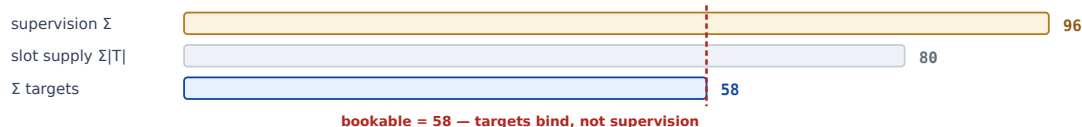


Figure 2 — Nested ceilings. Panel A: an advance booking may fill only 8 of a PGY-2's 16 Monday slots; the gap is held for urgent and discharge work. Panel B: on this real day the binding constraint is the sum of targets (58), not supervision (96) — supervision would only bind below $A = 58 / 32 \approx 1.8$, i.e. with a single attending.

§5 State: the continuity partition

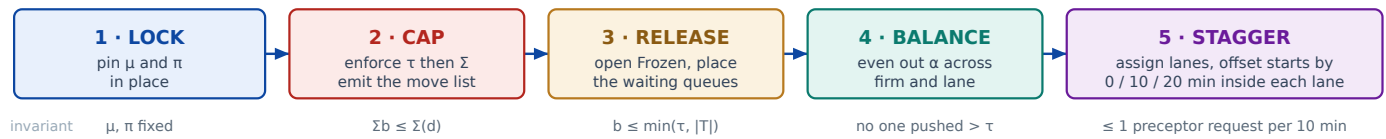
Each present resident carries a booked load split by **continuity level** — how tightly the patient is bound to that doctor. This partition is what makes a principled overflow decision possible:

$$b(r) = \mu_r + \pi_r + \alpha_r$$

μ C1 — **must see own doctor**. Immovable. · π C2 — **prefer own team**. Movable to the firm partner only. · α C3 — **any doctor**. Freely movable.

A **firm** pairs one PGY-2 with one PGY-1 inside a named attending lane — the partial order that stages 4 and 5, and the whole call-out cascade, navigate.

§6 The five-stage pipeline



STAGE 2 — removal priority (lexicographic; stop as soon as $\Sigma b \leq \Sigma$)



Within a pass the victim is $\arg \max_r b(r)$ — the busiest resident — so repeated removal levels the remaining loads to within one patient of each other. Each displaced patient is quoted their doctor's own nextClinicDates, computed from §2, so the move list names a real destination rather than "reschedule".

Figure 3 — BUILD THE DAY. Five stages, each preserving the invariant printed beneath it. Stage 2 is the only stage that removes patients, and it exhausts the freely-movable pool before touching anything with a continuity claim.

6.1 Stage 2 in detail

Capping runs in two phases. The **local** phase brings each resident to their own target; the **global** phase then enforces supervision across the day.

(a) local for each r : while $b(r) > \tau(r)$, remove one from α , else from π ; never from μ .

A second loop enforces $b(r) \leq |T|$; only here may μ be broken, and doing so raises a named warning.

(b) global while $\Sigma_r b(r) > \Sigma(d)$: draw from the first non-empty pass of $\langle (PGY-2, \alpha), (PGY-1, \alpha), (PGY-2, \pi), (PGY-1, \pi) \rangle$, victim = $\arg \max b(r)$.

PGY-1 loads are protected ahead of PGY-2 — a deliberate policy choice: a PGY-1 has half the slots and a 60-minute teaching visit is the hardest thing in the day to re-place.

6.2 Worked example — the supervision squeeze

Six PGY-2s with 8 booked each and three PGY-1s with 4 each, on a Monday, with a **single** attending. All patients are "any doctor".

Quantity	Value	Derivation
Booked before	60	$6 \times 8 + 3 \times 4$
Per-resident targets	—	$8 \leq 12$ and $4 \leq 5$, so the local phase removes nothing
Supervision cap Σ	32	1 attending $\times$ 32
Excess	28	$60 - 32$
Drawn from pass 1	28	PGY-2 "any doctor" pool holds $48 \geq 28$, so passes 2-4 never run
PGY-1 touched	0	protected — their 12 bookings are untouched
Booked after	32	exactly the cap; the six PGY-2 loads level to 4, 4, 3, 3, 3, 3

This is acceptance test 6, asserted on every page load.

6.3 Cost

Each removal or placement re-filters and re-sorts the candidate pool, so with $n = |R(d)|$ residents present, an excess X and a waiting queue Q , building a day costs $O((X + |Q|) \cdot n \log n)$ with all loops explicitly bounded. Forward searches for a resident's next clinic date are capped at a 420-day scan, so they terminate inside the academic year by construction. The year heatmap evaluates $a(r, d)$ once per resident per clinic weekday — $40 \times 261 = 10,440$ evaluations, each $O(1)$ — which is why the whole year renders in a single pass with no caching.

WHAT THIS IS NOT

The pipeline is a **greedy heuristic with an explicit, auditable priority order**, not an optimiser. It does not compute a globally optimal assignment and makes no such claim. That is the intended trade: every decision it takes can be read back as one sentence in the move list and overruled by the scheduler, which matters more at a front desk than optimality.

§7 The call-out cascade

When resident g is absent their load (μ, π, α) must be redistributed *today*. Define the spare capacity of p when allowed x over target:

$$c(p, x) = \max(0, \min(\tau_p + x, |T_p|) - b_p)$$

The overflow allowance x is itself clipped by the template maximum, so **no relaxation can ever produce more patients than there are slots**. Only one call site uses $x > 0$.

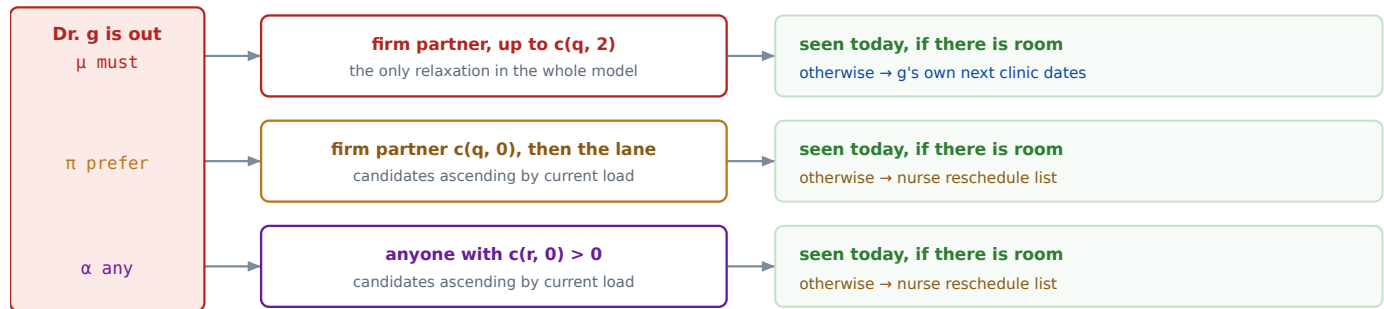


Figure 4 — Redistribution under absence. Each continuity class is offered the tightest acceptable destination first and spills only when that destination is genuinely full. Only C1 patients are allowed to push a partner over target, and only by two.

CONSERVATION

No patient is created or lost by the cascade:

$$\mu + \pi + \alpha = \text{placed}_{\text{must}} + \text{placed}_{\text{prefer}} + \text{placed}_{\text{any}} + \text{deferred to } g\text{'s next dates} + \text{nurse list}$$

This identity is asserted directly in acceptance test 7, so a redistribution that silently dropped a patient would fail the build.

§8 The application surface

Five screens, each a thin presentation of one function above.

Screen	Computed by	Question answered
1 · Who's in clinic?	residentsOnDate, residentsUndecidedOnDate, computeCapacity	Who can I book — and who don't we know about yet?
2 · When can I book Dr. X?	adviseBooking, nextClinicDates, prevClinicDates	Real dates in a window, ranked by remaining ceiling
3 · Year at a glance	renderHeatmap over $a(r, d)$	Which days months ahead are thin, which crowded?
4 · Build today's day	buildDay, layoutDay, renderGrid	Make today fit, and tell me exactly what to move
5 · Someone called out	sameDayFallback, attendingDropped	Where does each patient go now?

Tab 2 ranks candidate dates by remaining ceiling κ – booked, breaking ties toward the earliest. The application stores **no patient-level data of any kind** — every input is a count. Only configuration is persisted, and only to the browser running it.

§9 Verification and honest limits

9.1 Executable specification

Nine acceptance tests run on every page load. They are the specification:

#	Asserts
1	block partition; friendly failure outside the year
2	O / N / E / A resolve correctly across four blocks
3	only outpatient residents appear before days are set
4	Thursday AM only; Friday 09:00; lunch never used
5	an empty window still returns nearest dates
6	the 60 → 32 squeeze; PGY-1 protected
7	cascade bound $\tau+2$ and the conservation identity
8	all 261 weekdays render with counts and names
9	an undecided clinic day is never guessed or counted

9.2 What the model does not claim

- The rotation matrix C is **programme-supplied data**. The tool cannot verify it and does not try.
- δ is empty until entered. Coverage of the year is therefore a function of scheduler effort, not of the algorithm.
- τ , ρ and $p = 32$ are **policy parameters**, not measured capacities. All are editable.
- Because only counts are held, no true panel-level continuity rate can be computed — continuity is enforced *prospectively*, per booking.
- Default firm pairing is by roster order and should be replaced with the real firm structure.
- No optimality guarantee: §6 is a heuristic with a stated priority order.

Provenance. Every quantity in this document is produced by the shipped implementation — one self-contained HTML file with no dependencies. Figure 2B (80 slots, 58 target, 96 cap, 38 ceiling) is the application's own output for Monday 31 August 2026 with three attendings; §6.2 and the §7 bound are acceptance tests 6 and 7. Nothing here is estimated, benchmarked or projected.